

MCP 연동을 통한 '도구 확장'

에이전트가 우리 파일·문서·데이터에 직접 손댈 수 있도록
표준 연결 방식을 익힌다

에이전트는 어떻게 우리와 일할 수 있는가?

Day 3 · Module 1 ·

09:00–11:00

MCP 구조 이해 — 클라이언트-서버와 권한 범위

좋은 에이전트/도구의 역할, 6가지 질문에 답할 수 있어야 합니다

도구 이름·설명·스키마가 구체적일수록 에이전트가 올바른 기능과 값을 선택한다

도구 설명이 구체적일수록 선택 정확도와 운영 안전성이 높아진다



언제 쓰는가

사용 조건과 제외 조건



필수 입력

필드·형식·허용 범위



무엇을 반환하는가

결과 형식과 근거 ID



어떤 부작용인가

파일·레코드·외부 상태 변경



실패 시 오류

원인·재시도·복구 정보



권한과 승인

읽기·쓰기·외부 실행 게이트

현재 여러분의 MCP 경험은 1.6%다

사전 설문(16명 응답) 기준 — 오늘은 용어를 안다는 전제 없이 처음부터 시작한다

81.3%

AI를 거의 매일 사용

17.7%

코딩 에이전트 경험

1.6%

MCP 서버 연결 경험

오늘의 원칙

그래서 오늘은 코드 한 줄 없이, 이미 만들어진 MCP 서버를 "연결"하는 것부터 시작한다.

새 앱을 스마트폰에 설치하는 것과 비슷한 수준의 작업이라고 생각하면 된다.

MCP를 이해하면 AI Agent의 ‘손과 발’이 보입니다

표준 연결 × 도구 엔지니어링 × 권한·승인 × 제조 적용

MCP 를 통해서 AI Agent 의 연결 구조와 통제 지점을 설명할 수 있습니다

개념 → 작동 원리 → 아키텍처 → 통신 → 보안 → 제조 적용 → 실습 순으로 진행합니다.

구간	핵심 질문	학습 결과
개념·원리	Agent는 왜 도구가 필요한가?	모델·하네스·도구를 구분
MCP 구조	누가 누구와 무엇을 주고받나?	구성요소와 호출 흐름 설명
보안·협업	어디서 멈추고 승인받나?	권한·A2A·거버넌스 판단
적용·실습	제조 현장에 어떻게 시작하나?	PoC 청사진과 통제표 작성

MCP(Model Context Protocol)란 무엇인가

MCP는 AI와 외부 시스템을 공통 형식으로 연결하지만, 연결만으로 안전한 자동화가 완성되지는 않는다

비유로 설명하면

옛날 휴대폰은 기종마다 충전 단자가 달랐지만, USB-C가 생기며 하나의 규격으로 여러 장치를 연결할 수 있게 됐다.

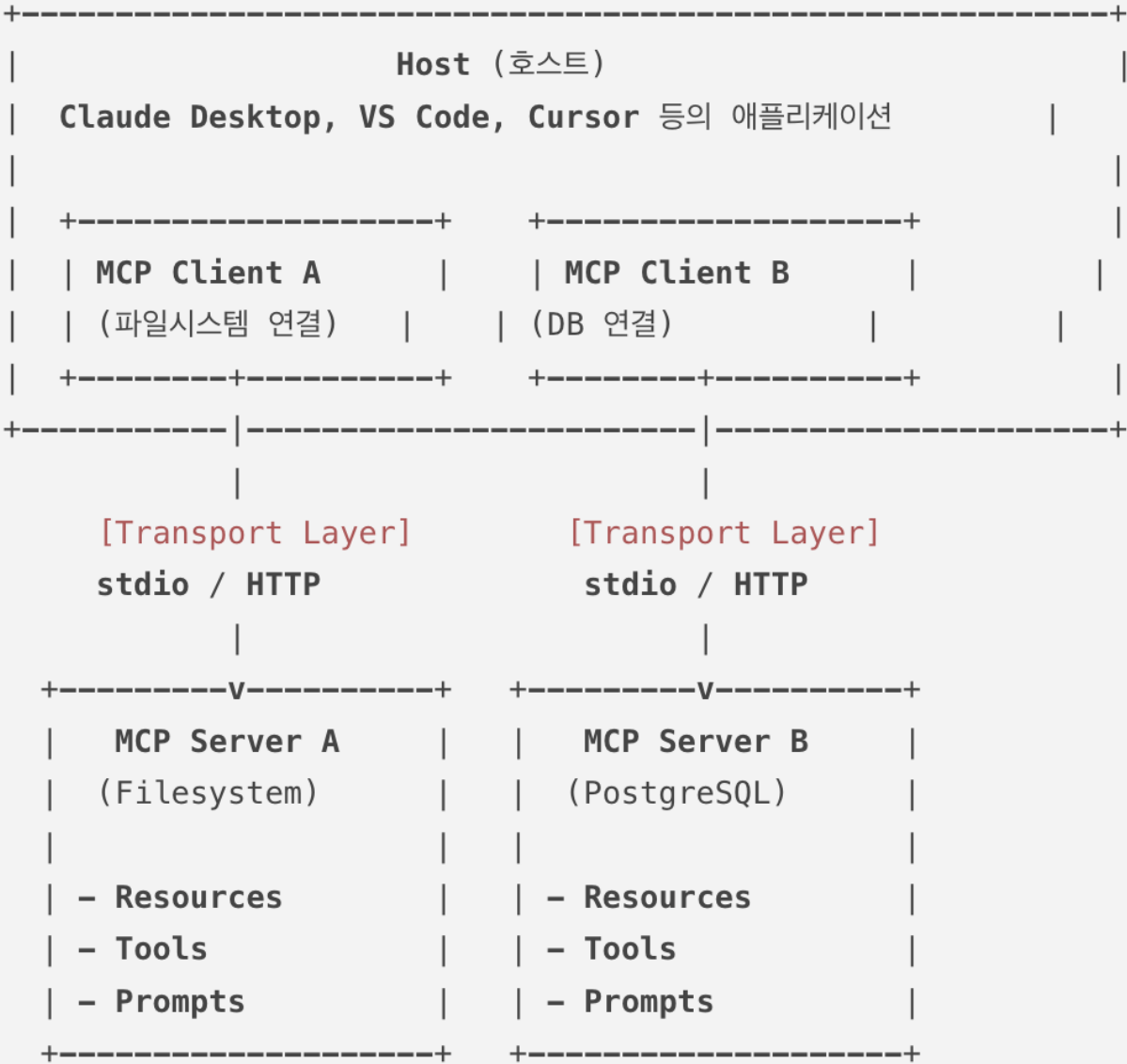
MCP는 AI판 USB-C다. 파일·DB·GitHub·브라우저·사내 API를 같은 방식으로 발견하고 호출하게 한다. 다만 연결 가능성, 실행 권한, 업무 절차, 최종 책임은 별도로 설계해야 한다.

[MCP\(Model Context Protocol\)](#)는 AI 도구가 외부 서비스와 소통하는 표준 프로토콜입니다. [GitHub](#)에 이슈를 만들고, 데이터베이스를 조회하고, 디자인 파일을 읽는 일이 하나의 규격 안에서 이루어집니다. 이 장에서는 MCP의 동작 원리를 이해하고, 다양한 환경에서 MCP를 설정하는 방법, 그리고 직접 MCP 서버를 만드는 과정까지 단계별로 안내합니다.

Model Context Protocol (MCP)는 [Anthropic](#)(Claude 개발사)이 주도하여 발표한 오픈소스 표준 프로토콜입니다.

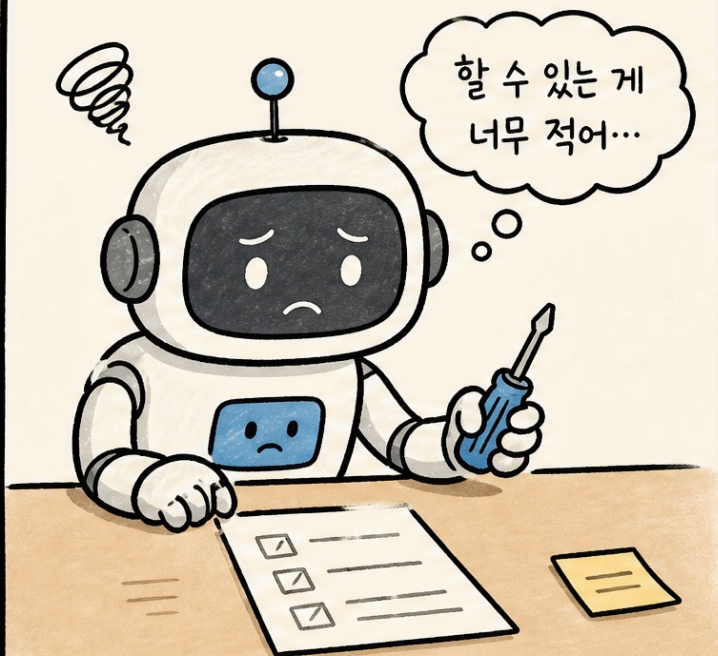
간단히 말해, **AI 모델(또는 AI 코딩 도구)과 외부 데이터 원본(또는 도구)이 서로 안전하고 규격화된 방식으로 대화하기 위한 범용 인터페이스 표준**입니다.

MCP 시스템은 세 계층 구조로 동작합니다.

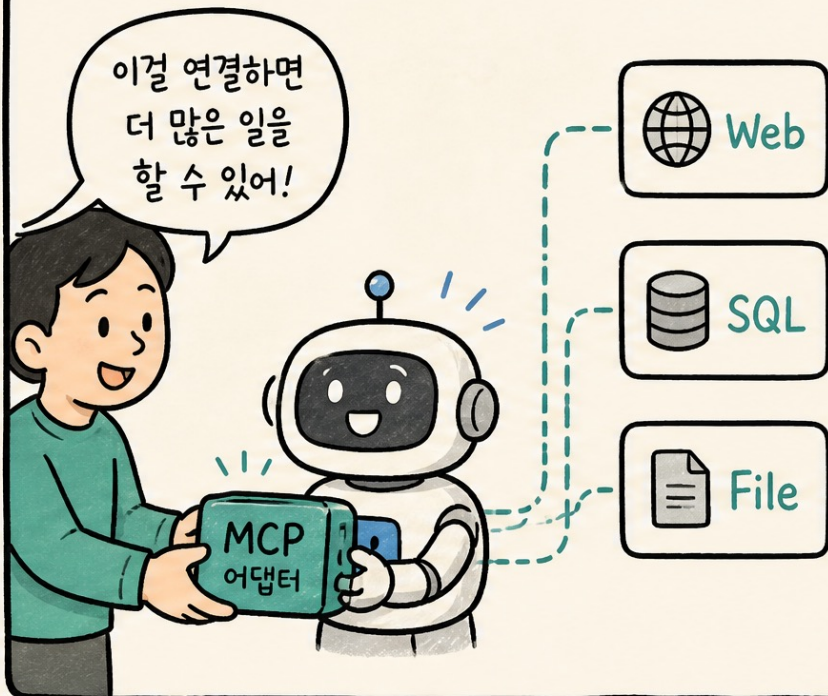


✨ MCP로 확장하는 에이전트 능력 ✨

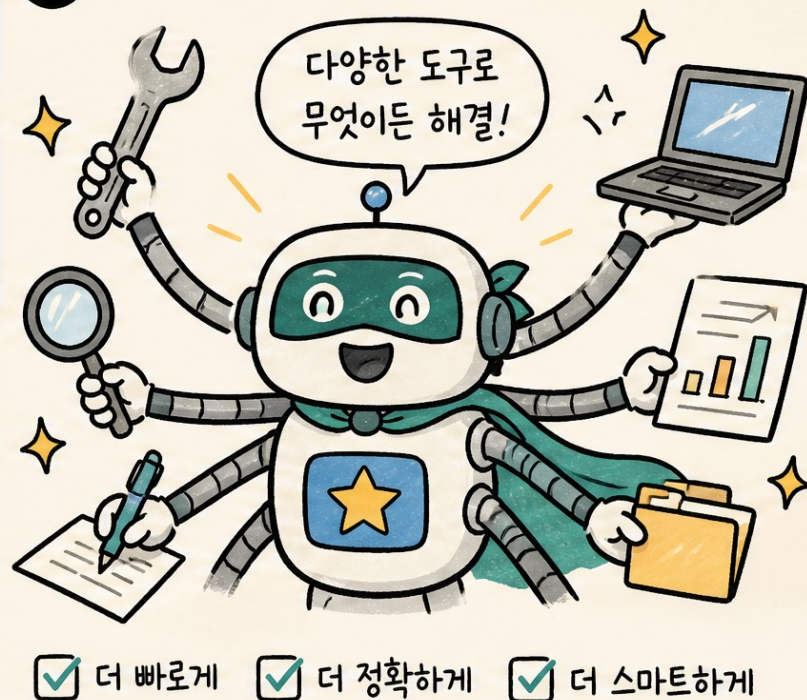
1 제한된 도구로는 한계가 있어요...



2 MCP 어댑터로 새로운 능력을 연결해보자!



3 이제 나는 슈퍼 워커!



MCP 이전과 이후, 무엇이 달라 질 수 있을까?

전용 연결을 반복하는 대신 공통 규격으로 도구를 발견하고 호출한다 — 인증·오류·권한은 여전히 운영 설계가 필요하다

MCP 이전

도구마다 각자 다른 연결 코드

파일시스템 연동, 스프레드시트
연동을 각각 다른 방식으로
구현해야 했다.

도구를 하나 추가할 때마다
새로운 연결 방법을 배워야 했다

MCP 이후

표준 규격 하나로 여러 도구

서버만 바꾸면 새 도구가 연결된다.
마치 스마트폰에 앱을 설치하듯 필요한 서버를
"추가"만 하면 된다.

오늘 우리가 할 일이 바로 이것
<https://github.com/modelcontextprotocol/servers>

업무 데이터가 시스템 마다 분산될수록 ‘연결 방식’에 병목이 발생합니다

기존 통합

앱마다 별도 어댑터

AI 앱 'A'가 PLM·MES·QMS에 연결되고, AI 앱 'B'도 같은 시스템에 다시 연결됩니다.

인증·오류·데이터 형식이 통합마다 달라 유지보수 부담이 커집니다.

연결 개수가 늘수록 변경 영향도 함께 커집니다.

표준 연결

공통 규격으로 연결

MCP는 호스트와 외부 기능 사이의 메시지·기능 발견·호출 규칙을 표준화합니다.

표준은 모든 비즈니스 로직을 없애는 것이 아니라, 연결 계약을 공통화합니다.

USB-C 비유: 포트는 같아도 장치의 기능은 다릅니다.

LLM Agent는 ‘생각 → 행동 → 관찰 → 수정’을 반복합니다

01

목표 이해

사용자 요청과 성공 기준
을 해석

02

도구 선택

검색·파일·API 중 필요한
행동 결정

03

결과 관찰

도구 결과와 오류를 다시
읽음

04

계획 수정

다음 호출 또는 최종 답변
결정

챗봇은 주로 ‘말’하지만, Agent는 외부 상태를 읽거나 바꾸는 ‘행동’까지 수행합니다.

Agent = Model + Harness: 성능은 모델 밖에서도 결정됩니다

자동차 Wiring Harness

전력·신호를 안전하게 전달

전선을 묶고 보호하며 경로와 인터페이스를 표준화합니다. 고장 지점을 추적하고 진동·열·과전류 위험을 줄입니다.

부품이 좋아도 연결이 나쁘면 차량은 불안정합니다.

AI Agent Harness

컨텍스트·도구·권한을 운영

모델 호출, 도구 실행, 메모리, 오류 처리, 승인, 검증, 로그를 하나의 실행 환경으로 묶습니다.

MCP는 하네스 전체가 아니라 ‘도구 연결층’의 일부입니다.

MCP와 A2A는 하네스의 서로 다른 계층을 다룬다

≡ 같은 여행 예약 요청도, 도구 연결과 역할 협업은 다르게 설계한다 ≡



요청: 두바이 항공권 예약하고 캘린더에 추가해줘

MCP: 도구 연결 계층

- 도구 설명
- 입력 스키마
- 권한 확인
- 오류 처리
- 결과 형식



하네스 에이전트

MCP 서버

MCP 서버

항공편 검색
좌석 예약
결제 확인



항공권 API



캘린더 API

일정 생성
시간 확인
초대 발송

경쟁
관계가
아니라
조합
관계

A2A: 에이전트 협업 계층

- 역할 분담
- 위임 메시지
- 상태 공유
- 결과 통합
- 재시도 방식



오케스트레이터
(팀장 에이전트)



항공권 에이전트



일정 에이전트

상태 공유
(진행 상황)

위임:
항공권 예약해줘

결과:
예약 완료!

결과:
예약 완료!

결과:
일정 추가 완료!

통합 답변



- 공유 작업 목록
- ☒ 항공권 검색
 - ☒ 좌석 예약
 - ☒ 캘린더 추가
 - ☐ 알림 발송



하네스 관점: MCP는 에이전트가 도구를 쓰는 방법, A2A는 에이전트들이 함께 일하는 방법이다

도구는 함수 목록이 아니라 ‘안전한 실행 계약(Contract)’입니다

좋은 도구 레이어는 등록부터 결과 포매팅까지 실행 전후를 함께 설계합니다.

책임	쉬운 정의	제조·하드웨어 비유	실무 질문
등록	무슨 도구가 있는지 알림	공구함 인덱스	언제 써야 하는가?
스키마 검증	입력 칸·형식 확인	치수·전압 규격	필수 값이 맞는가?
권한 확인	행동 위험도 판정	작업 허가서	누가 승인해야 하는가?
격리 실행	제한된 환경에서 실행	시험 셀·방호벽	실패 피해를 가둘 수 있는가?
결과 포매팅	모델이 읽기 좋게 정리	검사 성적서	다음 판단에 필요한 정보인가?

도구 호출은 모델의 요청과 시스템의 실행이 분리됩니다

01

요청

‘설비 A의 최근 알람을 보여줘
,

02

호출 생성

`get_alarm(asset_id='A')`

03

정책·실행

입력 검증 후 시스템이 API 실행

04

결과 반영

알람 목록을 관찰해 설명 생성

핵심: 모델은 행동을 ‘제안’하고, 하네스가 권한·검증을 거쳐 실제로 ‘집행’합니다.

Function Calling, Tool Calling, MCP는 같은 말이 아닙니다

개념	무엇인가	주요 책임	비유
Function Calling	정의된 함수를 구조화해 호출	이름·인자·결과	특정 장비의 전용 커넥터
Tool Calling	검색·코드·브라우저 등 행동 호출의 상위 개념	선택·실행·관찰	공구를 골라 작업
MCP	도구·데이터·프롬프트를 표준 방식으로 노출	발견·협상·메시지·수명주기	공통 포트와 통신 규격

좋은 도구 이름·설명·스키마가 모델의 선택 정확도를 높입니다

모호한 설계

do_stuff(data)

- 언제 써야 하는지 불명확
- 입력 의미와 단위가 없음
- 읽기/쓰기 위험이 섞임
- 출력 형식과 오류가 불명확

명확한 설계

lookup_order_status(order_id)

- 주문 조회 상황에만 사용
- 주문번호 형식 명시
- 읽기 전용으로 선언
- 상태·예정일·갱신시각 반환

버튼 이름이 '작업 1'인 제어반과 같습니다.

도구 설명은 사람용 홍보문이 아니라 모델용 작업 표지판입니다.

도구가 많아지면 능력보다 '혼란'이 먼저 늘 수 있습니다

버튼 200개 리모컨

모든 도구를 항상 노출

비슷한 이름과 긴 설명이 컨텍스트를 소비합니다. 잘못된 도구 선택 경로와 위험 표면이 함께 늘어납니다.

보고서 Agent에게 DB 삭제 도구는 필요 없습니다.

안내 데스크

필요한 도구를 지연 로딩

현재 단계에 필요한 읽기 도구부터 보여주고, 작성·전송 단계에서 도구 집합을 바꿉니다.

도구 검색은 '찾기', 권한은 '사용 허가'입니다.

도구가 많을수록 좋은 에이전트가 되는 것은 아니다

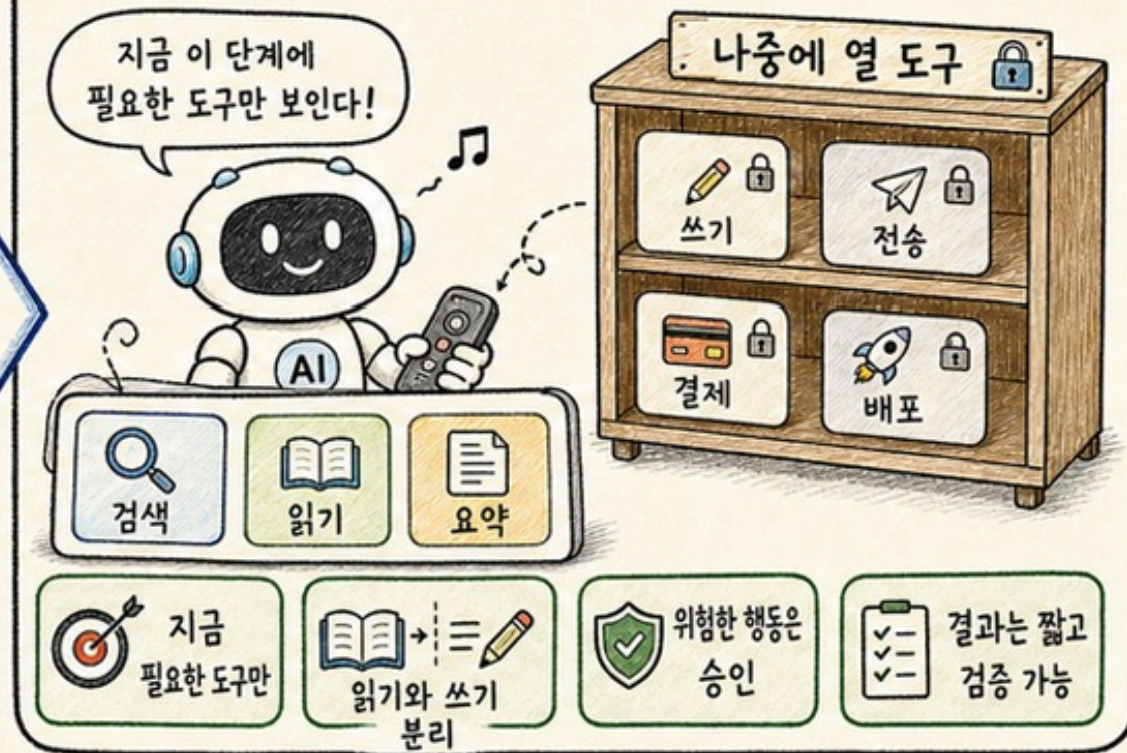
≡ 버튼 200개 리모컨보다, 지금 필요한 버튼만 보이는 하네스가 낫다 ≡

나쁜 예: 도구 폭발



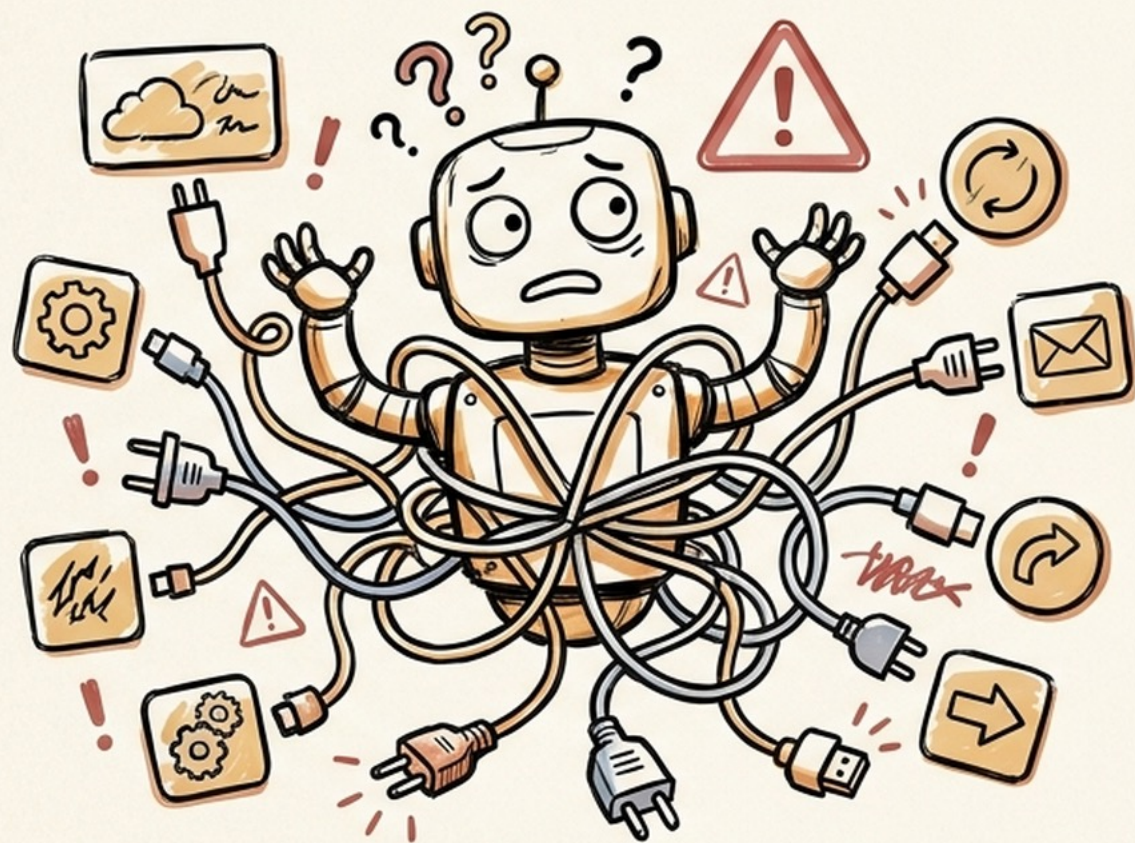
도구를 많이 주기보다,
필요한 순간에
필요한 만큼
보여준다

좋은 예: 단계별 도구 노출

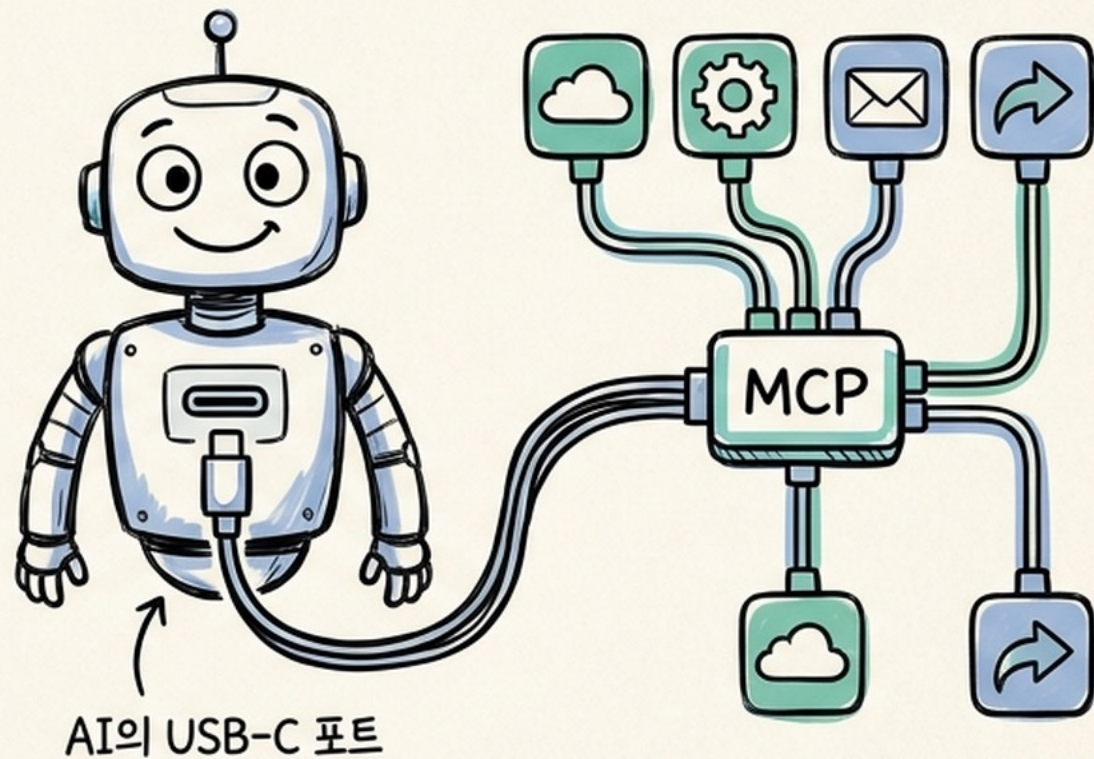


하네스 관점: 도구 범위는 성능 문제이자 안전 문제다

도구마다 따로 붙이지 말고
MCP 하나로 연결한다



제각기 커스텀 연동 · 매번 새로



표준 규격 · AI의 USB-C 포트

MCP는 AI 애플리케이션과 외부 기능 사이의 공통 연결 규격입니다

쉬운 정의

AI용 USB-C

서로 다른 AI 앱과 도구·데이터 소스가 공통 규격으로 연결됩니다.
. 새 호스트나 새 서버를 조합하기 쉬워집니다.

비유의 한계

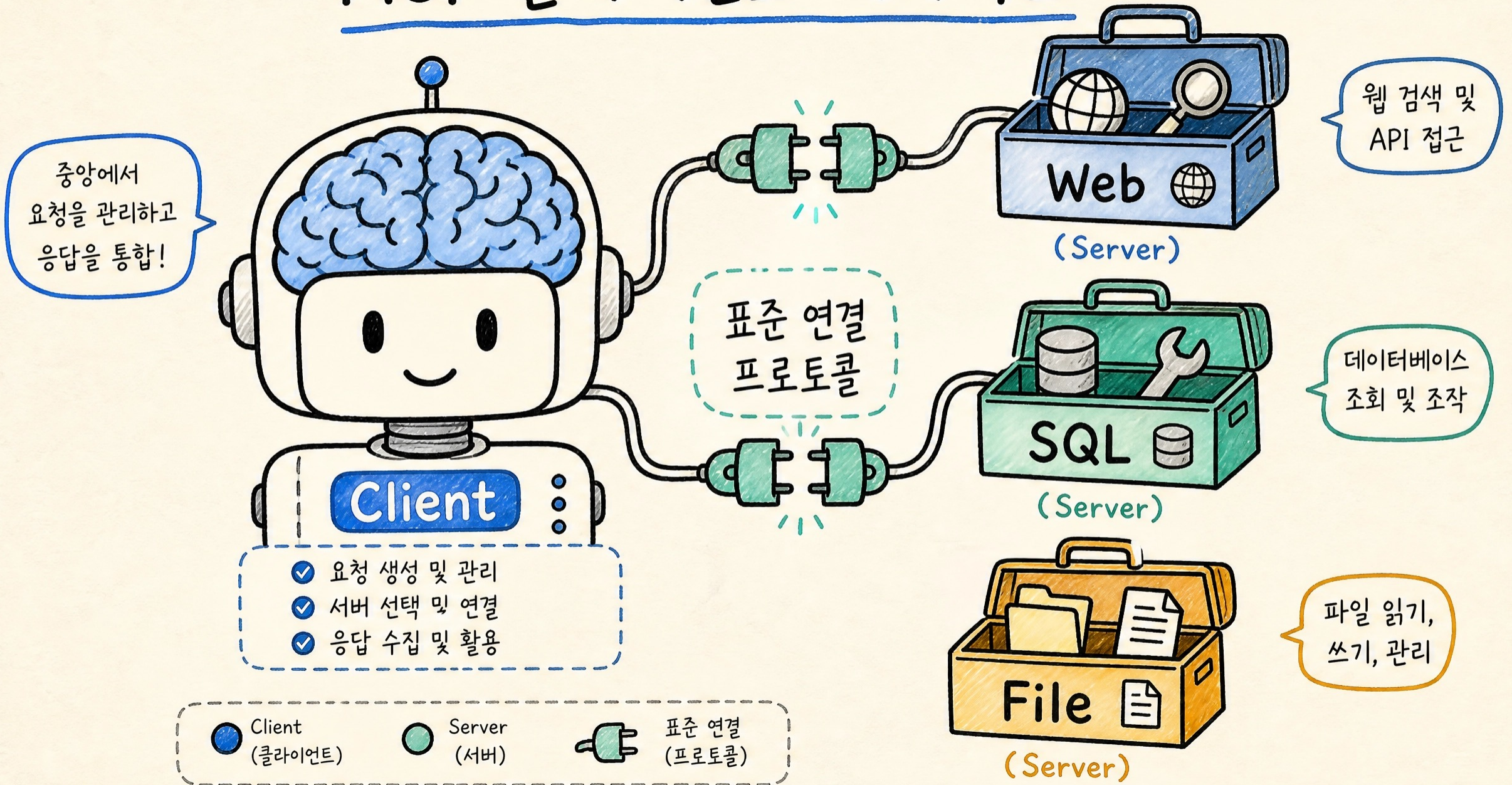
포트만 꽂는다고 안전한 것은 아님

MCP는 연결 규격이지 업무 권한, 데이터 품질, 책임 체계 전체를 자동 해결하지 않습니다. 호스트 정책과 서버 신뢰가 필요합니다.

원문 비유: 다양한 장치를 표준 포트에 꽂는 방식

호환성 ≠ 신뢰성 ≠ 승인

MCP 클라이언트-서버 구조



Host가 정책을 잡고, Client가 1:1 연결을 유지하며, Server가 기능을 제공합니다

01

Host

사용자 UI·LLM·정책·승인·컨텍스트 통합

02

Client

특정 서버와 1:1 세션·협상·메시지 라우팅

03

Server

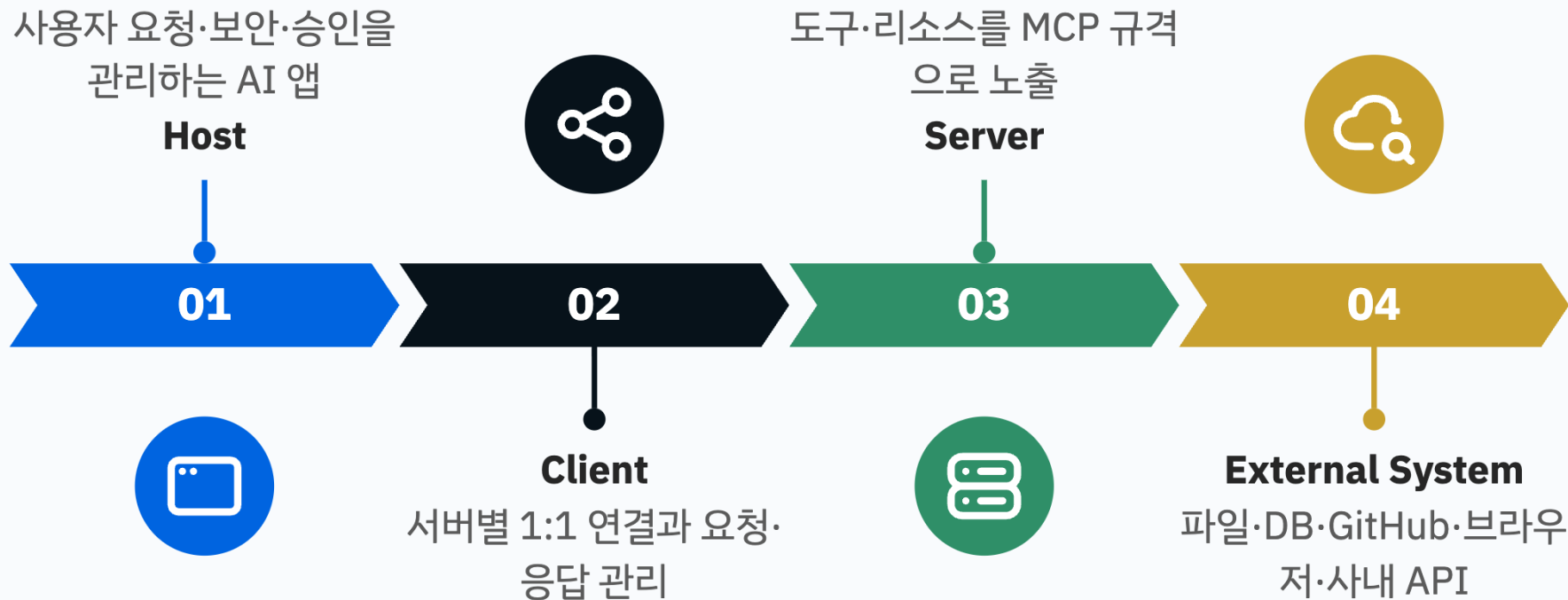
도구·리소스·프롬프트를 집중된 책임으로
노출

제조 비유: Host=라인 관제실, Client=전용 통신 모듈, Server=설비 기능 게이트웨이

MCP는 4가지 주체가 역할을 나눠 연결한다

Host가 보안과 승인을 조정하고, Client·Server를 거쳐 실제 업무 시스템에 접근한다

네 주체가 역할을 나눠 AI와 실제 업무 시스템을 연결한다



Server는 Tools·Resources·Prompts를 서로 다른 통제 방식으로 노출합니다

같은 서버 안에 있어도 ‘행동’, ‘맥락’, ‘템플릿’은 통제 주체와 위험이 다릅니다.

Primitive	쉬운 정의	주요 통제	제조 예시
Tools	모델이 호출할 수 있는 실행 기능	모델 선택 + Host 승인	알람 조회, 작업지시 초안 생성
Resources	읽어올 수 있는 맥락 데이터	애플리케이션이 첨부·선택	매뉴얼, 검사기준, 설비 이력
Prompts	사용자가 선택하는 재사용 템플릿	사용자 주도	8D 보고서 질문 템플릿

연결은 초기화·역량 협상 후에야 실제 기능 호출로 넘어갑니다

01

연결

stdio 또는 HTTP 통신 채널
생성

02

초기화

프로토콜 버전과 구현 정보 교
환

03

역량 협상

도구·리소스·샘플링 지원 범위
확인

04

운영

요청·응답·알림·취소·로그 처
리

하드웨어 비유: 전원 인가 → 핸드셰이크 → 지원 명령 확인 → 정상 운전

메시지는 JSON-RPC 2.0, 운반은 **stdio** 또는 Streamable HTTP가 담당합니다

로컬 연결

stdio

Host가 로컬 MCP 서버 프로세스를 실행하고 표준 입력/출력으로 JSON-RPC 메시지를 주고받습니다. 로컬 파일·개발 도구 연결에 적합합니다.

원격 연결

Streamable HTTP

독립 실행 서버의 단일 HTTP 엔드포인트를 통해 POST/GET과 선택적 스트리밍을 사용합니다. 네트워크 인증·보안이 중요합니다.

같은 제어반 내부의 직결 배선에 가깝습니다.

공장 간 네트워크 게이트웨이에 가깝습니다.

도구 호출은 발견 → 선택 → 승인 → 실행 → 관찰로 이어집니다

01

발견

Client가 서버의 도구
목록과 스키마 확인

02

선택

모델이 목적에 맞는 도
구와 인자 제안

03

승인

Host가 정책·사용자
확인으로 실행 허가

04

실행

Server가 기능을 수행
하고 결과 반환

05

관찰

모델이 결과·오류를
읽고 다음 행동 결정

실패는 ‘모델 문제’ 한 가지가 아니라 계층별로 분리해야 합니다

실패 계층	예시	주요 원인	대응
모델 판단	잘못된 도구 선택	설명 모호·도구 과다	설명 개선·도구 범위 축소
입력 스키마	asset_id 누락	필수값·타입 오류	실행 전 검증·재질문
권한	쓰기 도구 거부	승인·역할 부족	사람 승인 또는 안전한 대안
통신	시간초과·연결 끊김	네트워크·서버 장애	제한 재시도·중단·알림
업무 규칙	종료 설비 변경 요청	정책·상태 불일치	업무 오류로 명확히 반환

기존 API와 MCP의 차이는 ‘기능’보다 ‘연결 범위’에 있습니다

관점	일반 API 통합	MCP 통합	핵심 해석
대상	애플리케이션 간 기능 호출	LLM Host와 컨텍스트·도구 연결	MCP는 AI 통합에 초점
발견	API 문서·SDK를 별도 해석	역량·도구 목록을 프로토콜로 교환	동적 조합에 유리
수명주기	서비스별 구현	초기화·협상·세션 규칙 제공	공통 하네스 구축에 유리
업무 로직	API 내부에 존재	대개 기존 API를 서버가 감쌘	MCP가 API를 없애지 않음

MCP는 하네스의 도구 레이어이며, 목표·검증·기록까지 대신하지 않습니다

01

목표·컨텍스트

무엇을 왜 해야 하는가

02

MCP 도구층

무엇을 읽고 실행할 수 있는가

03

권한·오류

어디서 막고 어떻게 복구하는
가

04

검증·기록

결과가 맞고 추적 가능한가

MCP 연결 성공은 PoC의 시작 조건이지, 업무 품질과 안전의 완료 조건이 아닙니다.

‘권한’은 ‘모델의 판단’과 ‘시스템의 집행’을 분리하는 장치입니다

01

Allow

낮은 위험의 읽기·검색을 자동 허용

02

Ask

수정·전송·실행 전 사용자에게 영향 설명

03

Deny

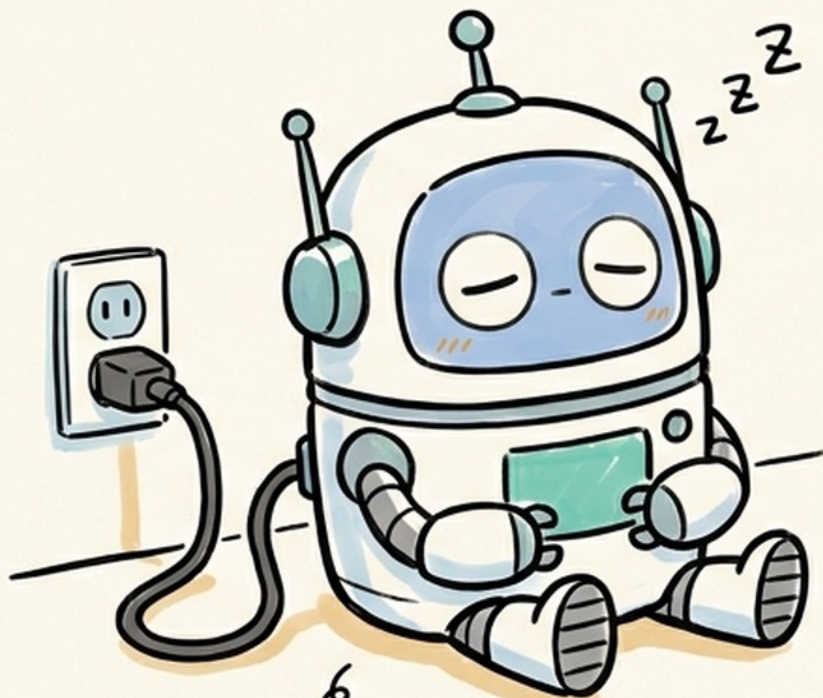
삭제·비밀 접근·범위 밖 행동을 정책으로 차단

승인 화면에는 ‘어떤 도구가, 어떤 인자로, 무엇을 바꾸는지’가 보여야 합니다.

(예)AX 진행시 주요 권한은 ‘변동성·외부 영향·안전 영향’으로 나눌 수 있습니다

행동	예시	기본 정책	추가 통제
읽기	매뉴얼·알람·이력 조회	자동 허용 가능	범위·민감도·마스킹
초안	작업지시·보고서 초안	자동 생성 + 검토	원문 링크·불확실성 표시
업무 기록 수정	QMS 코멘트·티켓 상태	사람 승인	변경 전후 diff·복구
외부 전송	협력사 메일·공식 보고	책임자 승인	수신자·첨부·보안등급 확인
물리 제어	PLC 설정·설비 정지	기본 차단	별도 안전 시스템·현장 절차

에이전트도 사람도
지금은 충전 시간

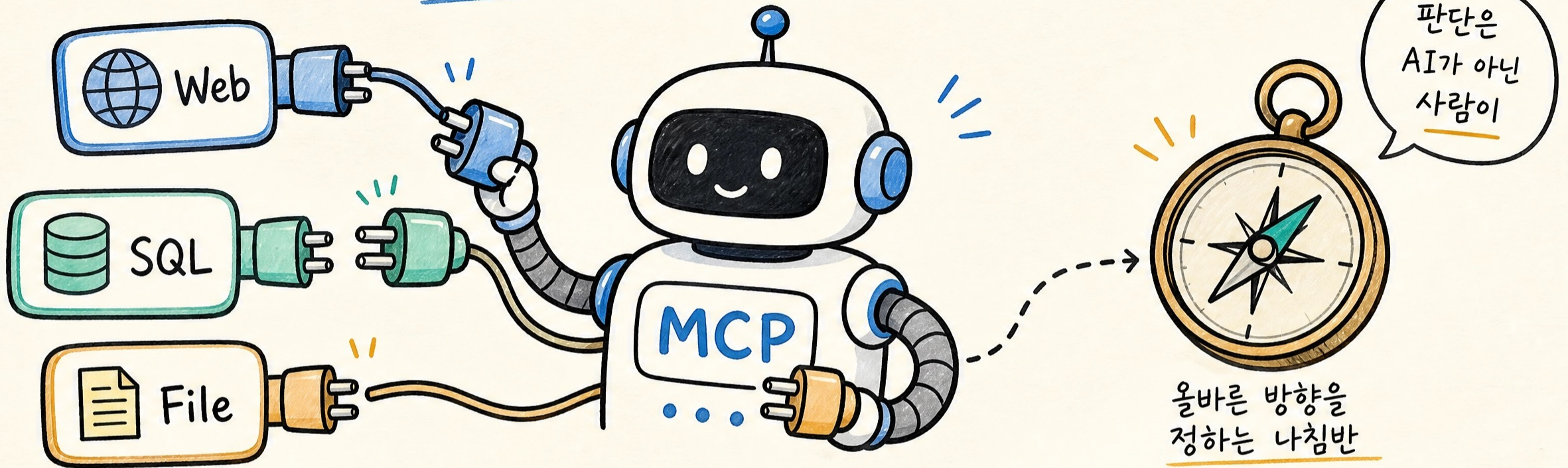


6
생각도 배가 고파다

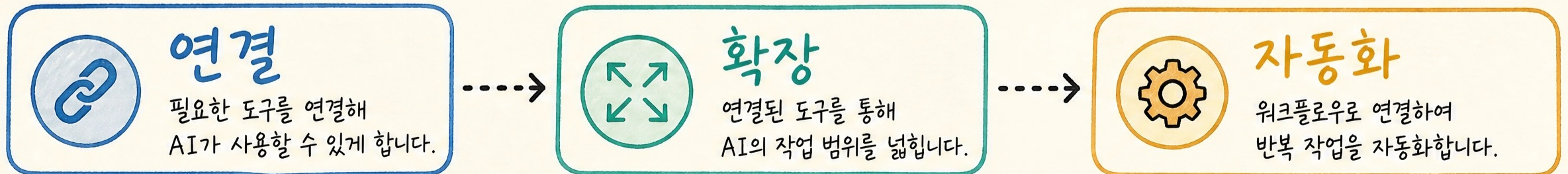


점심시간 · 휴식 12:00 ~ 13:00

≡ AI의 작업 범위를 확장하는 ≡



MCP 도구 연동과 워크플로우 ☆



레스토랑에 빗대면

낮선 용어 3개(Host·Client·Server)가 한 번에 안 들어온다면 이 비유로 감을 잡는다

비유로 설명하면

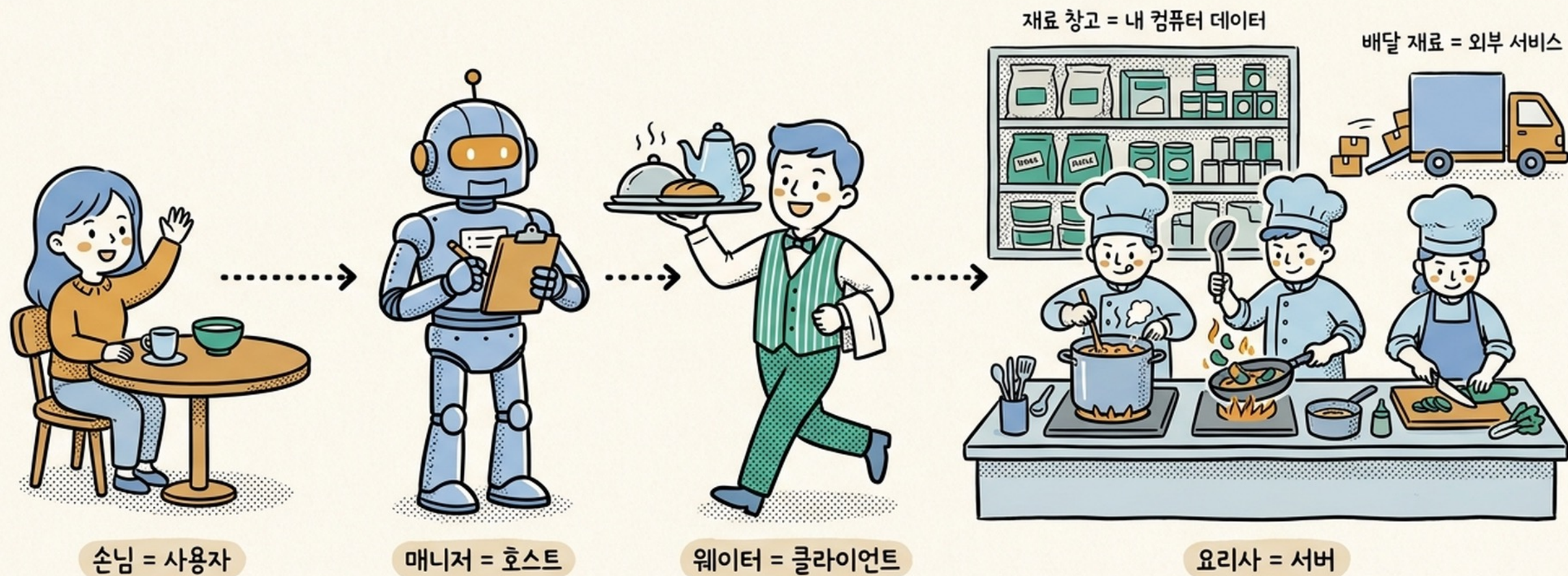
레스토랑에서 식사하는 상황을 떠올려보자.

Host(여러분) — 손님이 "이 요리 주세요"라고 주문한다.

Client(웨이터) — 손님의 주문을 정확히 주방에 전달하고, 완성된 요리를 다시 손님에게 가져다준다.

Server(주방) — 실제로 요리를 만든다. 무슨 재료가 있는지, 무슨 메뉴가 가능한지는 주방이 가장 잘 안다.

식당으로 이해하는 MCP 구조 한눈에 보기



MCP로 무엇을 할 수 있나

활용 분야 4가지



데이터 관리

로컬 문서 자동 정리
드라이브 회의록 분석



웹 자동화

실시간 시장 조사
뉴스 요약 생성



생산성 향상

슬랙 협업 자동화
일정 · 경로 정리

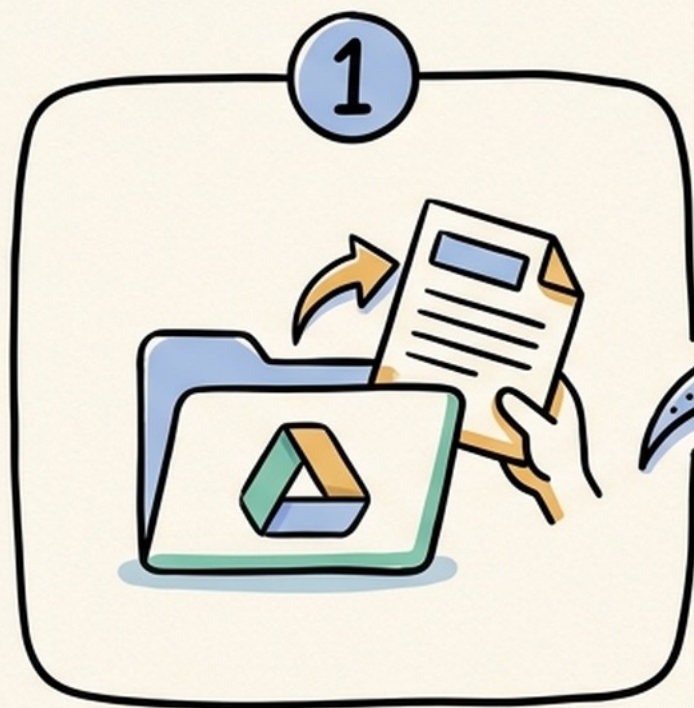


서비스 통합

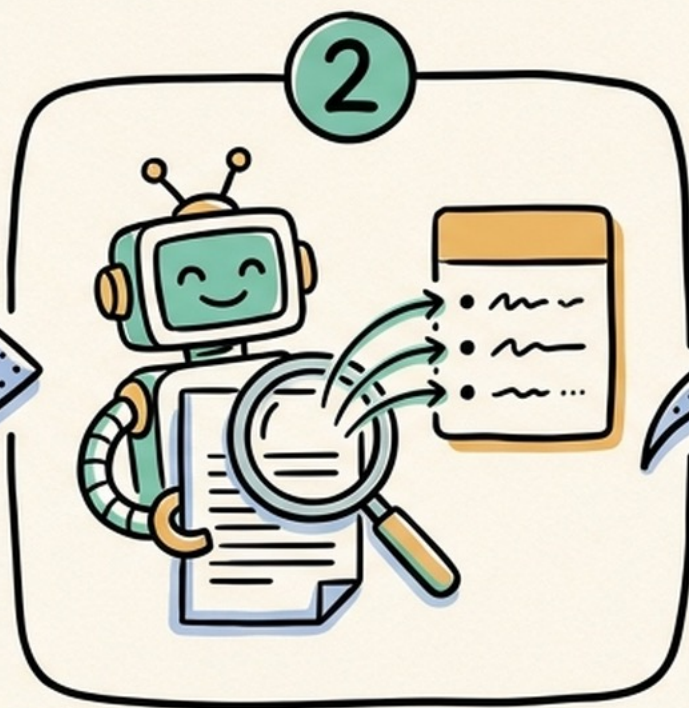
데이터베이스 조회
결제 · 배포 관리

대표 예제 — 회의록 자동 공유

드라이브와 메신저를 이어붙이기



회의록 불러오기



핵심만 요약하기



팀 채널에 공유

서버 두 개를 연결하면 한 문장으로 끝난다

비개발자도 할 수 있다

MCP 시작 4단계

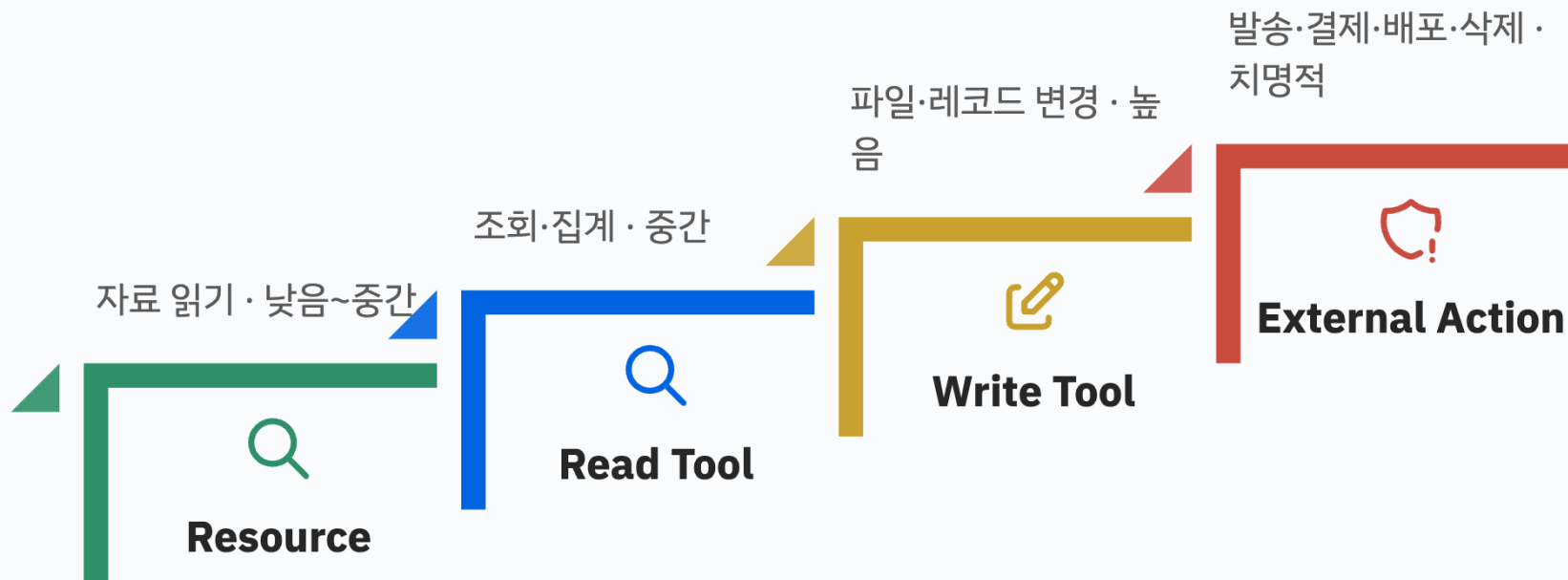


첫 실습은 웹 검색 서버부터

읽기와 실행은 다른 위험등급이다

Resource → Read Tool → Write Tool → External Action 순으로 위험과 승인 강도가 커진다

연결 설계에서는 읽기와 실행을 분리하고 위험이 커질수록 승인을 강화한다



MCP 보안은 서버 신뢰·데이터 경계·토큰 경계를 함께 봐야 합니다

위협	어떻게 발생하나	통제 원칙	현장 질문
악성/오염된 서버	도구 설명·결과가 오도	신뢰 목록·검토·격리	누가 서버를 배포했나?
과도한 데이터 공유	필요 이상 맥락 전송	최소 공개·동의·마스킹	서버가 볼 데이터는 어디까지?
토큰 탈취·오용	로그·캐시에 자격증명 노출	안전 저장·단기 토큰·대상 검증	토큰이 특정 서버에 묶였나?
프롬프트 인젝션	문서·웹 내용이 행동 유도	데이터와 지시 분리·승인	외부 문서가 정책을 덮을 수 있나?

거버넌스는 권한 ‘표’에 ‘기록·평가·운영 지표’를 더한 체계입니다

01

정책

허용·승인·차단과 역할 정의

02

증거

호출·인자·결과·승인자 감사 로그

03

평가

정확성·안전성·회귀 테스트

04

운영

비용·지연·실패·사고 대응

좋은 시스템은 실패를 숨기지 않고, 빨리 발견하고 책임 있게 복구할 수 있게 만듭니다.

MCP는 Agent→도구, A2A는 Agent↔Agent 협업을 표준화합니다

관점	MCP	A2A	비유
연결 대상	Agent/Host와 도구·데이터	독립 Agent 시스템끼리	공구 사용 vs 팀 협업
핵심 단위	Tools·Resources·Prompts	Capability·Task·Message·Artifact	명령/자료 vs 업무 위임
내부 노출	서버 기능 계약 노출	상대 Agent 내부 상태는 숨길 수 있음	장비 인터페이스 vs 협력사 계약
함께 사용	Agent가 MCP 도구를 장착	A2A로 다른 Agent에 업무 위임	작업자가 공구를 들고 팀과 협업

제조 현장의 MCP는 현장 시스템을 ‘읽기 중심의 (안전한) 컨텍스트’부터 시작합니다

01

문서

SOP·매뉴얼·도면 메타
데이터

02

상태

MES 실적·설비 알람·품
질 이력

03

분석

원인 후보·관련 변경·점
검 항목

04

초안

작업지시·8D·변경 검토
초안

05

승인

사람이 검증 후 시스템
기록

OT 제어 명령은 별도의 기능안전·사이버보안 절차 없이 MCP Agent에 직접 위임하지 않습니다.

(가설) 설비 이상 대응 Agent는 알람을 ‘정비 판단 맥락’으로 바꿀수 있다

읽기 단계

알람 + 매뉴얼 + 정비 이력

MCP 서버가 설비 ID 기준으로 최근 알람, 관련 매뉴얼 절, 유사 고장 이력을 조회합니다. Agent는 가능한 원인과 점검 순서를 근거 링크와 함께 정리합니다.

쓰기 단계

작업지시 ‘초안’ 생성

필요 부품, 안전 확인, 점검 절차를 초안으로 만들되 CMMS 등록은 정비 책임자가 승인합니다. 실제 설비 제어는 분리합니다.

자동 허용 후보: 범위가 제한된 읽기 도구

승인 대상: 작업지시 등록·부품 주문·설비 제어

도입은 '읽기 전용 1개 업무'에서 시작해 통제와 평가를 넓힙니다

01

업무 선정

빈도 높고 읽기 중심인
좁은 문제

02

서버 설계

명확한 도구·스키마·데
이터 경계

03

통제 설계

허용·승인·차단·감사 로
그

04

평가

대표 사례·오류·안전 회
귀 테스트

05

확장

검증 후 제한적 쓰기와
새 시스템 추가

PoC 성공 기준은 '연결됨'이 아니라 '정확하고, 안전하며, 추적 가능한 업무 결과'입니다.

Agent에게 ‘절대 자동화하면 안 되는 행동’은 무엇입니까?

판단 축 3가지

가역성 · 영향 · 책임

1. 되돌릴 수 있는가?
2. 외부 또는 물리 세계에 영향을 주는가?
3. 책임자가 명확한가?

세 질문 중 위험 신호가 클수록 자동화보다 승인·차단이 우선입니다.

조별 합의: ‘기본 차단’ 행동을 2개 선정하십시오.

행동 후보

허용·승인·차단으로 분류

- 알람·매뉴얼 조회
- 작업지시 초안 저장
- 정비 기사 자동 배정
- 설비 정지 명령
- 협력사 데이터 발송

각 행동의 승인자와 완료 증거까지 정하십시오.

참고 용어

Risk appetite · Human oversight · Fail-safe · Accountability

운영 전 체크리스트: 연결보다 먼저 책임과 증거를 확인하십시오

영역	반드시 답할 질문	완료 증거
업무	목표·사용자·성공 기준이 한 문장인가?	승인된 Use case 정의
데이터	소유자·민감도·전송 범위가 명확한가?	데이터 맵·마스킹 규칙
도구	이름·설명·입력·오류가 명확한가?	도구 계약·테스트
권한	누가 무엇을 허용·승인·차단하는가?	권한 매트릭스·승인 UI
운영	로그·평가·복구·사고 대응이 가능한가?	Trace·Eval·Runbook

MCP의 핵심은 '더 많이 연결'이 아니라 '표준화하고 통제하며 검증'하는 것입니다

01

표준화

Host와 도구·데이터의 연결 계약

02

분리

모델 판단과 시스템 집행의 경계

03

통제

최소 권한·승인·데이터 경계

04

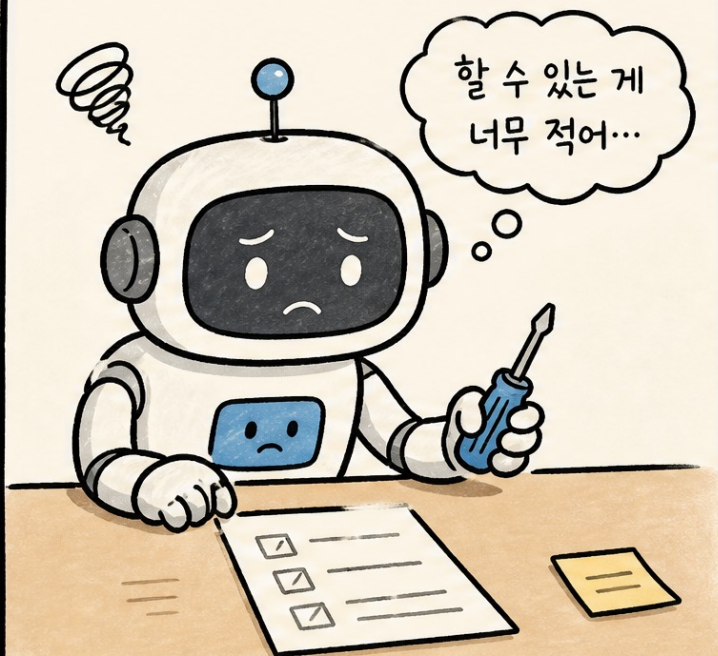
검증

결과 상태·감사 로그·회귀 평가

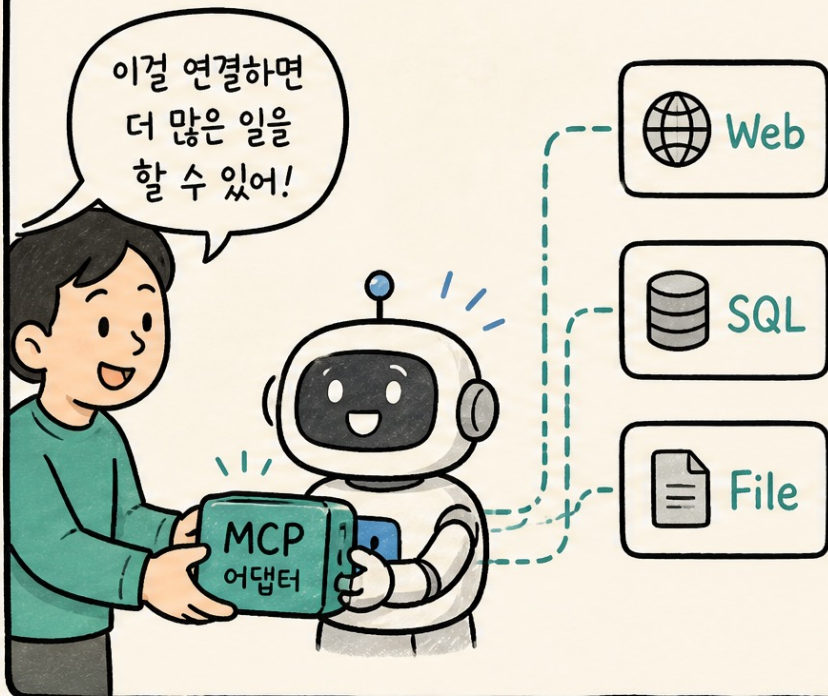
첫 행동: 우리 조직의 '읽기 전용 1개 업무'와 '사람 승인 1개 지점'을 오늘 정의해보십시오.

✨ MCP로 확장하는 에이전트 능력 ✨

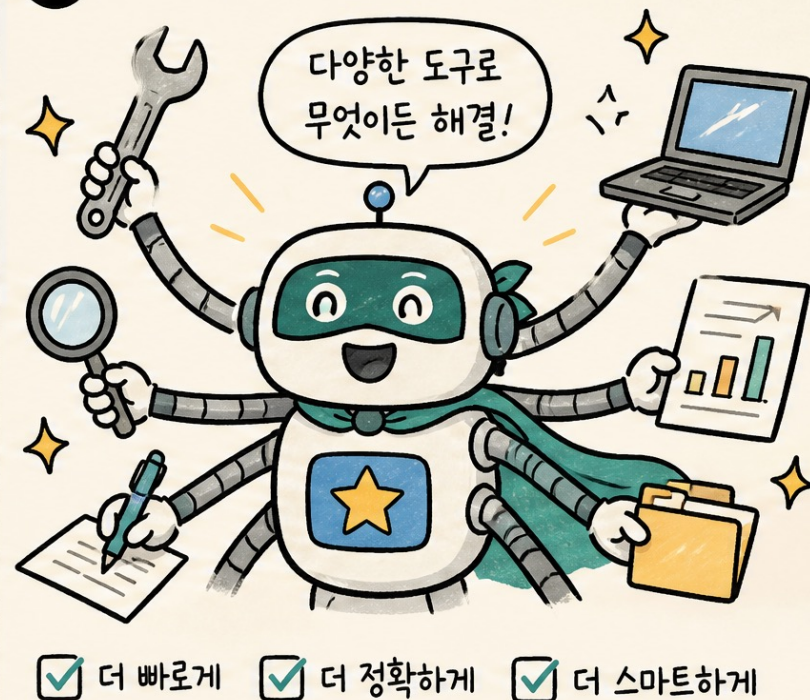
1 제한된 도구로는 한계가 있어요...



2 MCP 어댑터로 새로운 능력을 연결해보자!



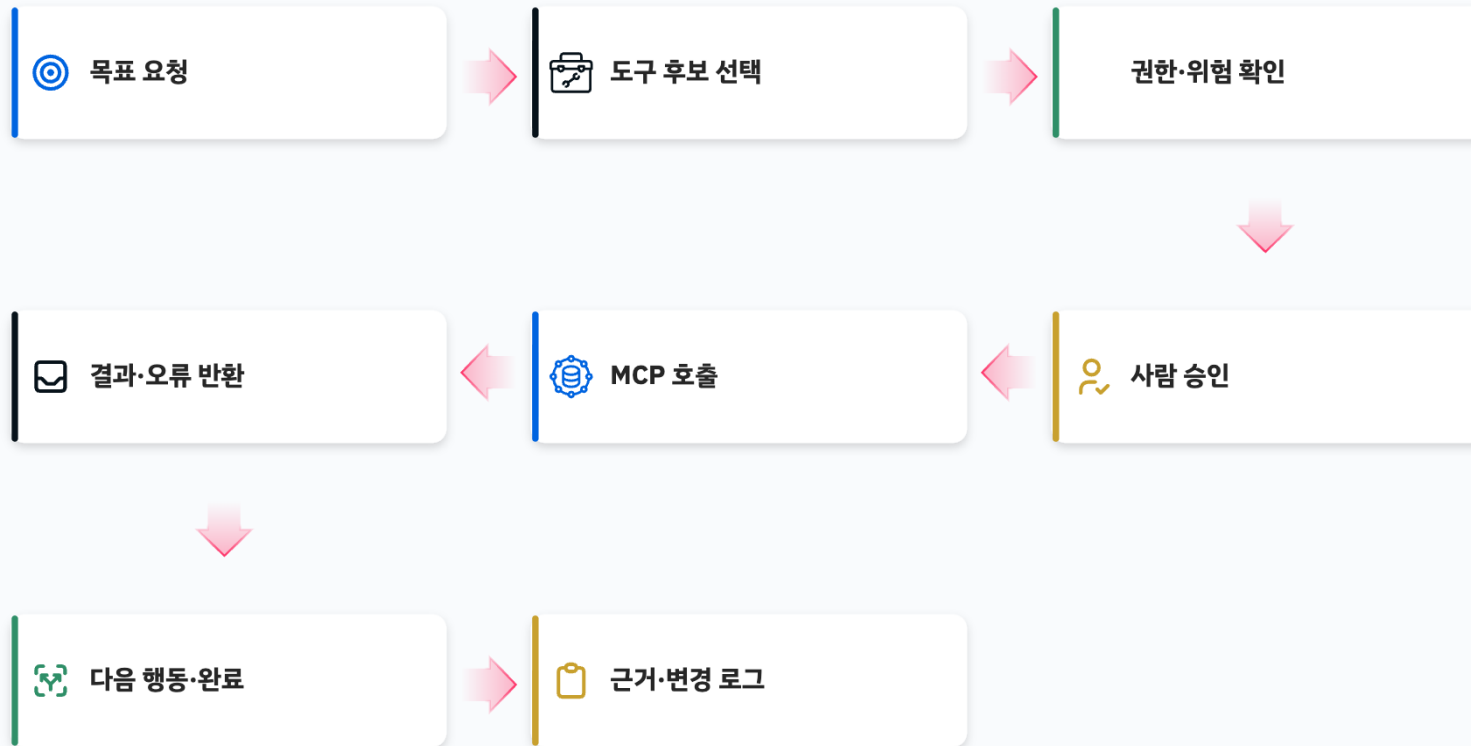
3 이제 나는 슈퍼 워커!



MCP 실행은 선택부터 로그까지 8단계다

도구 호출 전후의 권한·승인·결과·변경 증거까지 포함해야 업무 실행이 완성된다

도구 호출 전후의 권한·승인·증거까지 포함해야 업무 실행이 완성된다



최소 권한·비밀정보·승인 경계를 함께 설계한다

MCP는 도구 연결 계층이며, 접근 범위·인증·승인·로그는 하네스가 별도로 통제한다

● 승인된 읽기 범위

폴더·저장소·레코드 단위로 제한 — 첫 단계 기본값

● 비밀정보 분리

토큰은 환경변수·보안 저장소에 두고 설정·로그에 남기지 않는다

● 쓰기·외부 실행

변경 범위를 미리 보여주고 사람 승인 후 실행한다

비유로 설명하면

신입사원에게 회사 마스터키를 주지 않는 것과 같다.

읽기 → 초안 쓰기 → 내부 수정 → 외부 실행 → 삭제·관리 순서로

검증된 범위만 단계적으로 연다.

막혔을 때

자주 겪는 문제와 해결 순서

MCP가 처음이라면 이 세 가지에서 대부분 막힌다

증상	해결 순서
"그런 서버가 없다" 오류	서버 이름 오타를 확인한다 — <code>claude mcp list</code> 로 연결된 서버 목록을 먼저 본다
파일이 안 보인다	<code>--scope project</code> 로 연결했는지, 폴더 경로가 정확한지 확인한다
검색 결과가 이상하다	질문을 더 구체적으로 바꿔서 다시 요청한다 ("오류" → "ERR-204 오류 코드")

나만의 MCP 서버 만들기

기성품 MCP 서버만으로도 강력하지만, 진짜 힘은 **나만의 MCP 서버를 직접 만들 때** 발휘됩니다. 사내 API, 내부 데이터베이스, 독자적인 워크플로우를 MCP 서버로 감싸면, AI 에이전트가 여러분의 업무 환경 전체를 이해하고 조작할 수 있게 됩니다.

1. 왜 직접 만들어야 하는가?

- **사내 시스템 연동**: 회사 내부의 REST API, GraphQL 엔드포인트, 레거시 시스템을 AI가 직접 호출할 수 있게 됩니다.
 - **맞춤형 워크플로우**: "Jira 티켓 생성 후 Slack 알림 발송" 같은 복합 작업을 하나의 도구로 묶을 수 있습니다.
 - **데이터 접근 제어**: 외부 SaaS에 데이터를 보내지 않고, 로컬에서 안전하게 데이터를 처리하는 중간 계층을 만들 수 있습니다.
 - **지능의 확장**: 공개 서버에 없는 우리 팀만의 특수한 도구를 AI에게 즉시 학습시킬 수 있습니다.
-

2. Python으로 5분 만에 만들기 (FastMCP)

Python의 **FastMCP** 라이브러리를 사용하면 복잡한 설정 없이 단 몇 줄의 코드로 MCP 서버를 구축할 수 있습니다.

단계 1: 환경 준비 및 라이브러리 설치

초고속 패키지 매니저인 **uv** 사용을 권장합니다.

```
uv init my-mcp-server  
cd my-mcp-server  
uv add mcp
```

단계 2: 서버 코드 작성 (`main.py`)

데코레이터 기반의 직관적인 코드로 도구를 정의합니다.

```
from mcp.server.fastmcp import FastMCP

# 서버 이름 설정
mcp = FastMCP("MyFirstServer")

@mcp.tool()
def add_numbers(a: int, b: int) -> int:
    """두 숫자를 더합니다."""
    return a + b

@mcp.tool()
def get_user_status(username: str) -> str:
    """사용자의 현재 상태를 조회합니다. (예시 API)"""
    # 실제 환경에서는 DB나 사내 API를 여기서 호출합니다.
    return f"{username}님은 현재 '바이브 코딩' 중입니다!"

if __name__ == "__main__":
    mcp.run()
```

💡 **Tip:** 함수의 `docstring` 이 에이전트가 이 도구를 언제 쓸지 결정하는 지침이 됩니다. 최대한 자세히 적으세요.

3. TypeScript로 정교하게 만들기

대규모 프로젝트나 복잡한 타입 정의가 필요하다면 공식 MCP SDK를 사용합니다.

단계 1: 프로젝트 초기화

```
mkdir my-ts-mcp && cd my-ts-mcp  
npm init -y  
npm install @modelcontextprotocol/sdk zod  
npm install -D typescript @types/node
```

단계 2: 서버 코드 작성 (`src/index.ts`)

```
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";
import { z } from "zod";

const server = new McpServer({ name: "my-ts-server", version: "1.0.0" });

server.tool(
  "calculate_bmi",
  "체질량지수(BMI)를 계산합니다.",
  {
    weight: z.number().describe("몸무게 (kg)"),
    height: z.number().describe("키 (cm)"),
  },
  async ({ weight, height }) => {
    const heightInMeters = height / 100;
    const bmi = weight / (heightInMeters * heightInMeters);
    return {
      content: [{ type: "text", text: `계산된 BMI는 ${bmi.toFixed(2)}입니다.` }],
    };
  }
);

async function main() {
  const transport = new StdioServerTransport();
  await server.connect(transport);
}

main().catch(console.error);
```

4. 디버깅: MCP Inspector 활용

서버를 도구에 연결하기 전, 웹 브라우저에서 직접 테스트해 볼 수 있는 강력한 도구입니다.

```
# Python 서버 테스트
npx @modelcontextprotocol/inspector python main.py

# TypeScript 서버 테스트
npx @modelcontextprotocol/inspector node dist/index.js
```

실행 후 브라우저에서 <http://localhost:3000>에 접속하면, 내가 만든 도구가 목록에 뜨고 직접 값을 넣어 실행 결과를 확인할 수 있습니다.

5. Claude Code에 연결하기

이제 내가 만든 서버를 실제 에이전트에게 장착시킵니다.

```
# 로컬 서버 등록
claude mcp add my-server -- python /절대경로/main.py

# 등록된 서버 확인
claude mcp list
```

이제 Claude에게 "**내 몸무게 70kg에 키 180cm인데 BMI 좀 내 서버로 계산해줘**"라고 말해 보세요. 에이전트가 여러분이 직접 만든 도구를 사용하여 답할 것입니다.

6. 배포와 공유

- **npm/PyPI 배포:** 패키지로 만들어 누구나 `npm` 나 `uvx` 로 실행하게 할 수 있습니다.
- **Smithery.ai:** 완성된 MCP 서버를 [Smithery](#)에 등록하여 전 세계 개발자들과 공유하세요.

오늘 배운 3가지

1

MCP 구조

Host·Client·Server·External System 네 주체의 역할을 구분하면 연결 구조가 보인다

2

권한 설계

연결 가능성과 실행 권한은 다르다 — 읽기와 쓰기를 분리하고 외부 실행은 승인한다

3

도구 제작

좋은 도구는 입력·출력·부작용·오류·권한을 계약으로 정의하고 운영지표로 검증한다

Day 3 인계 — `request_handoff.py --day 3` → 사람 승인 → `check_day_gate.py --enter-day 4`

지식 자산화 · 하네스 엔지니어링 실무

DAY 4 에서 뵙겠습니다

오늘도 애쓰셨습니다.